

CloudNativePG vs. Percona Operator for PostgreSQL: A Deep Architectural and Source-Level Comparison

TL;DR

- **The single defining difference is philosophical and structural:** CloudNativePG (CNPG) writes PostgreSQL HA logic natively into the operator and instance manager, using the Kubernetes API server as the sole source of truth, and deliberately rejects both StatefulSets and Patroni; the Percona Operator is a **hard fork of Crunchy Data's PGO v5** that delegates HA to **Patroni** (using the Kubernetes API as Patroni's DCS), uses **StatefulSets**, and drives backups with **pgBackRest**. As of mid-2026 the relevant releases are **CNPG 1.30.x** (CNCF Sandbox, Apache-2.0) and **Percona 2.9.0 / 3.0.0** (Apache-2.0, hard-forked from Crunchy PGO v5.8.x).
- **Choose CNPG** for Kubernetes-native purism, minimal operand images, plugin extensibility (CNPG-I), declarative multi-cluster DR with promotion tokens, and CNCF-governed open-source neutrality. **Choose Percona/PGO** for teams with deep Patroni + pgBackRest operational muscle memory, an existing Crunchy PGO estate, or a preference for the mature, battle-tested Patroni consensus model and Percona's vendor support subscriptions.
- Both are Go operators built on controller-runtime, both default to scram-sha-256, both support PostgreSQL 18, synchronous replication, PITR, connection pooling with PgBouncer, and cert-manager. The divergences are in *where the intelligence lives* and *how much machinery runs inside the database pod*.

Key Findings

1. **Lineage.** CNPG originated at 2ndQuadrant (first commit 18 Feb 2020 by Leonardo Cecchi, Marco Nenciarini, Gabriele Bartolini), was open-sourced by EDB under Apache-2.0 in May 2022, and was **accepted into the CNCF Sandbox on 21 January 2025** (some EDB/Bartolini posts cite 15 January 2025); it formally applied for Incubating status on 12 November 2025. Percona's operator began as a fork of Crunchy Data's PGO; version 0.1.0 was based on PGO 4.6.2, the 2.x line rebased onto **PGO v5** (Patroni-based), and **v3.0.0 (released 22 May 2026) completed a formal "hard fork"** from Crunchy PGO ~v5.8.x, moving inherited Crunchy CRDs to the upstream.pgvector.com API group (K8SPG-1007). [\(gabrielebartolini\)](#) [\(Percona\)](#)
2. **HA intelligence location.** CNPG: operator + instance manager (Go binary, PID 1). Percona: Patroni (Python) runs inside each database pod, coordinating through the Kubernetes API as its DCS. CNPG standardizes on promoting the replica with the highest LSN; Patroni uses leader-key/lease consensus with a watchdog/DCS-failsafe model. [\(gabrielebartolini\)](#)
3. **Pod management.** CNPG manages Pods and PVCs directly via a custom pod controller (no StatefulSets). Percona/PGO uses StatefulSets via the embedded Crunchy [\(PostgresCluster\)](#) reconciler.
4. **Backups.** CNPG has moved from in-tree barman-cloud (deprecated in 1.26, removal repeatedly pushed and now slated for 1.31 per the latest 1.29 release notes) to the CNPG-I **Barman Cloud Plugin** (gRPC sidecar) plus native volume-snapshot backups. Percona uses **pgBackRest** exclusively (embedded in the operand image), plus optional PVC-snapshot backups (2.9.0).
5. **Images and licensing.** Every image in the CNPG ecosystem is Apache-2.0 and the minimal operand image contains only PostgreSQL. Percona ships fully open-source, freely-redistributable images (a stated motivation for the hard fork, contrasting Crunchy's Developer Program restrictions), with community-built upstream images as a 3.0.0 tech preview (official in 3.1.0). [\(Percona\)](#)

Details

1. Project lineage, governance, and licensing

CloudNativePG. CNPG is the direct descendant of EDB/2ndQuadrant's proprietary "Cloud Native PostgreSQL." The public history begins with the first commit on 18 February 2020. EDB contributed the code to an open-source community under the CloudNativePG name in **May 2022** under the **Apache License 2.0**. After an earlier unsuccessful 2022 attempt, CNPG was **accepted to the CNCF at the Sandbox maturity level on 21 January 2025** per the CNCF project page (EDB's and Bartolini's own posts cite 15 January 2025) and formally applied to advance to Incubating on 12 November 2025. Governance is community/CNCF-based, though EDB remains the dominant contributor and offers commercial support (EDB Postgres AI / a certified operator for OpenShift). The project's scope is deliberately narrow and stated as project principles: **Kubernetes only** (vanilla, CNCF), **PostgreSQL only** (vanilla PGDG, no forks except via extensions/plugins), and **not a general-purpose database operator**. CNPG complies with the OpenSSF Best Practices Baseline and, at the **OpenSSF Security Slam 2026** (KubeCon EU Amsterdam), was, per Bartolini's May 2026 comparison, "one of only three projects to earn all five available badges (Cleaner, Chronicler, Inspector, Mechanizer and Defender)"; it publishes a SECURITY-INSIGHTS.yml and a Gemara threat assessment relevant to EU Cyber Resilience Act due diligence. [\(gabrielebartolini + 2\)](#)

Percona. Percona's operator is explicitly "based on Postgres Operator developed by Crunchy Data." The lineage: v0.1.0 built on PGO **4.6.2**; the modern 2.x series is built on **Crunchy PGO v5** (the Patroni + pgBackRest architecture). The migration guides note compatibility with Crunchy PGO up to **v5.8.6**. Per Percona's documentation, "Percona Operator for PostgreSQL 3.0.0 has been released on May 22, 2026. This version establishes Percona Operator for PostgreSQL as a hard fork of the Crunchy PGO project, making it fully independent." The mechanical

signature of the fork is CRD renaming: the inherited Crunchy CRDs (`PostgresCluster`, `PGUpgrade`, `PGAdmin`, etc.) were moved to a dedicated (`upstream.pgvector.percona.com`) API group (K8SPG-1007), enabling the Percona and Crunchy operators to coexist in the same cluster. Percona's own native CRDs under (`pgvector.percona.com/v2`) (e.g., `PerconaPGCluster`) were unchanged. The whole stack is Apache-2.0 and the images are freely redistributable — Percona's messaging frames this against Crunchy's Developer Program terms (which restrict production use for organizations with more than 50 employees and prohibit using the images to deliver support/consulting). Percona sells support subscriptions and consulting; there is also Percona Everest (a web UI) and community images (tech preview in 3.0.0, official in 3.1.0) built from upstream PostgreSQL packages. ([GitHub + 2](#))

Release cadence. CNPG produces builds per commit, cuts a minor roughly every two-to-three months (a new minor every three months is the stated target), supports the latest minor with a 3-month grace window, and uses release candidates. As of mid-2026 the newest series is **1.30** (with 1.29 and 1.28 still within support windows; 1.28 EOL 30 June 2026, 1.29 EOL 29 Sept 2026). Percona's cadence tracks Crunchy plus its own features; the 2.x line reached **2.8.1 (Dec 2025)** and **2.9.0 (31 March 2026)**, with **3.0.0 (code release 22 May 2026; OLM catalog 26 May 2026)**. Percona's CRD supports the last three minor operator versions. ([CloudNativePG](#)) ([CloudNativePG](#))

2. Architecture fundamentals

CNPG — the instance manager as PID 1. CNPG does not use any external failover manager. Each PostgreSQL pod runs the **instance manager**, a Go binary that is **PID 1** in the container and the parent of the (`postgres`) postmaster. The instance manager owns the full PostgreSQL lifecycle (start/stop via direct process control, promotion/demotion, `pg_rewind`, WAL archiving hooks), runs a local web server that backs the Kubelet's **startup, liveness and readiness probes**, and reports status (replication LSN/lag, timeline) directly to the Kubernetes API. The probes are database-aware: the startup and readiness probes use (`pg_isready`) (readiness can also gate on replica lag), and — from **1.27** — the primary's liveness probe performs an **isolation check**: if the instance manager can reach neither the API server nor any peer instance's REST endpoint, the probe deliberately fails so the Kubelet shuts the isolated primary down (a Kubernetes-native equivalent of Patroni 3.0's DCS failsafe mode). The (`.spec.switchoverDelay`) (default 3600s) bounds how long the old primary has to shut down gracefully and archive WAL. The operator explicitly detects WAL-disk-full and refuses to fail over in that case, preserving recoverability. ([gabrielebartolini](#))

Percona — Patroni, pgBackRest, PgBouncer. The Percona/PGO pod runs Patroni, which manages the postmaster and performs leader election and failover. Patroni uses the **Kubernetes API as its DCS** (endpoints/configmaps + leases), so there is no external etcd. HA in 2.9.0 uses **Patroni 4** (2.8.0 standardized on Patroni 4 as the only supported version, dropping the (`pgvector.percona.com/custom-patroni-version`) override). Backups are handled by **pgBackRest**, with dedicated **repo host** pods (StatefulSets) and repos targeting S3/GCS/Azure/PVC. Connection pooling is **PgBouncer**. The operator controllers are layered: the Percona (`PGClusterReconciler`) (`percona/controller/pgcluster`) translates the (`PerconaPGCluster`) CR into a Crunchy (`PostgresCluster`) (`v1beta1`) via (`PerconaPGCluster.ToCrunchy()`), and the embedded Crunchy (`internal/controller/postgrescluster`) reconciler materializes the StatefulSets, Services, Patroni config, and pgBackRest resources. Percona adds controllers for (`PerconaPGBackup`) (`percona/controller/pgbackup`) and restore/upgrade. ([Go Packages](#))

StatefulSets vs. direct Pod management. CNPG manages (`Pod`) and (`PVC`) objects directly. The rationale: StatefulSet ordinal identity cannot express "promote the pod with the highest LSN regardless of index," and coordinated configuration sequencing (e.g., raising (`max_connections`) on standbys before the primary) requires ordering logic the StatefulSet primitive does not provide; both are exactly the jobs a separate coordination layer (Patroni) does in PGO. The consequence for CNPG is finer control and a simpler in-pod footprint at the cost of re-implementing pod-lifecycle logic that StatefulSets give for free. Percona/PGO inherits StatefulSet semantics from Crunchy, which pairs naturally with Patroni's per-member identity model.

CRD design. CNPG's API group is (`postgresql.cnpg.io/v1`). Core CRDs: **Cluster** (the PostgreSQL cluster), **Pooler** (PgBouncer), **Backup**, **ScheduledBackup**, **ImageCatalog / ClusterImageCatalog**, **Database**, **Publication**, **Subscription**, and (1.30) **DatabaseRole**; plus the Barman Cloud plugin's **ObjectStore** CRD. Percona's native CRDs (`pgvector.percona.com/v2`): **PerconaPGCluster** (short name (`pg`)), **PerconaPGBackup**, **PerconaPGRestore**, **PerconaPGUpgrade** — sitting atop the inherited Crunchy **PostgresCluster** (now under (`upstream.pgvector.percona.com`)). Architecturally CNPG exposes one rich, first-class CRD surface; Percona exposes a Percona-facing CRD surface that is compiled down to Crunchy resources.

3. High availability and failover mechanics

CNPG failover. Failover is operator-driven and tracked in the Cluster status via (`currentPrimary`) and (`targetPrimary`). The sequence: the operator marks (`targetPrimary`) as pending, which forces the old primary pod to shut down so WAL receivers on replicas stop; the cluster enters a "Failing over" phase; once WAL receivers are stopped there is a leader election and the operator promotes the most-advanced replica (highest LSN); the former primary restarts, detects it is no longer primary, runs `pg_rewind`, and rejoins as a replica. This two-phase procedure prevents timeline divergence. In **1.30**, CNPG added a **Lease-based primary election** primitive: a Kubernetes Lease named after the cluster acts as a mutex that the instance manager must hold before acting as primary, released on clean shutdown so replicas can promote without waiting for the full TTL — an explicit split-brain guard. A 1.29-era fix also relabels a returning old primary "unhealthy" during the failover window so the (`-rw`) service does not route to it. Fencing is declarative: the (`cnpg.io/fencedInstances`) annotation (a JSON list, (`'[""]'`) for the whole cluster) shuts down the postmaster while keeping the pod running for inspection, disabling switchover/failover while set. ([GitHub](#))

([CloudNativePG](#))

Percona/Patroni failover. Patroni performs leader election through the Kubernetes DCS: a member must hold the leader key/lease before promoting; if it cannot renew, it demotes itself. Split-brain protection is three-layered — the Patroni layer (leader key race), the DCS layer (only one holder), and the OS layer (Linux watchdog / STONITH). Percona 2.9.0 added **standby WAL-lag detection**: when a standby exceeds a configurable lag threshold, the operator marks its pods unready and surfaces a `StandbyLagging` condition. Manual switchover is via `spec.patroni.switchover`. [Percona](#) [Percona](#)

Synchronous replication. CNPG offers two models. The legacy model (`minSyncReplicas`/`maxSyncReplicas`), quorum-based, pre-1.24, deprecated) auto-computes an integer `q` and self-heals by relaxing the requirement when replicas are unavailable. The modern `.spec.postgresql.synchronous` stanza (1.24+) supports **method: any** (ANY / quorum) and **method: first** (FIRST / priority), controls the full `synchronous_standby_names`, and adds a `dataDurability: required|preferred` knob (1.25) that chooses between strict durability (writes hang if quorum unavailable — PostgreSQL DBA-expected behavior) and self-healing. CNPG **1.28** promoted **Quorum-Based Failover** to stable via `spec.postgresql.synchronous.failoverQuorum`, which only promotes a new primary when a majority of synchronous standbys confirm — closing the "promote a lagging replica" data-loss window. (A known open issue notes that in *distributed* multi-cluster topologies the operator can still promote a lagging local replica; cross-cluster failover awareness is a documented limitation.) Percona exposes Patroni's `synchronous_mode`, `synchronous_mode_strict`, and `synchronous_node_count` through `patroni.dynamicConfiguration` — the classic Patroni synchronous model, applied without operator-side validation. [CloudNativePG](#) [Percona Community](#)

4. Backup and recovery

CNPG. CNPG's backup story reflects its Barman heritage (Barman originated in the 2ndQuadrant team). There are two supported approaches. (1) **Volume snapshots** — the only method handled directly by the operator — support hot and cold backups (hot requires the WAL archive for a consistent restore); Bartolini has demonstrated fully recovering "a 4.5TB Postgres database from a volume snapshot" in two minutes (benchmarked on AWS EKS r5.4xlarge nodes with a gp3 storage class). (2) **CNPG-I Barman Cloud Plugin** — the strategic direction. In-tree `barmanObjectStore` backup was deprecated in 1.26 (removal has been repeatedly pushed; the latest 1.29 notes slate it for 1.31). The plugin runs as a **gRPC sidecar** next to each instance; configuration lives in an **ObjectStore** CRD (auth/bucket in `.spec.configuration`), WAL settings under `.wal`, base-backup settings under `.data`; WALs go to a `wals/` directory and base backups as tarballs under `base/`. The model is **WAL-first continuous archiving** to an object store (S3/GCS/Azure), with full and PITR recovery via recovery targets. Bootstrap/recovery options: recovery from object store, `pg_basebackup` streaming bootstrap, and `bootstrap.initdb.import` (`pg_dump/restore`). The architectural point the plugin makes possible is **decoupling backup tooling from the operand image** — a `pgBackRest` or `WAL-G` plugin is possible under the same interface.

Percona. Percona is entirely **pgBackRest**-based, embedded in the operand image. It supports full/differential/**incremental** (incremental is the default), retention policies (`--repoN-retention-full/-diff/-archive`), scheduled (cron) and on-demand backups, and up to four repos across local PVC, S3, S3-compatible, GCS, and Azure. The primary is auto-configured for `pgbackrest archive-push` (WAL archiving) and replicas can be created via `pgBackRest` (`create_replica_methods`). PITR uses the WAL archive with a `restore_command` of `pgbackrest --stanza=db archive-get`. Restores/PITR are driven by the `PerconaPGBBackup`/`PerconaPGRestore` CRDs and a `dataSource` stanza for bootstrapping new clusters from a repo. 2.9.0 added **PVC-snapshot** backups (behind `BackupSnapshots=true`) for faster large-DB restores while still using `pgBackRest` for WAL consistency. [Percona + 3](#)

Philosophical contrast. CNPG's model is WAL-first streaming to object storage with the operator (or a plugin) orchestrating; the coupling of backup code to database code is broken by CNPG-I. Percona/PGO's model is the mature `pgBackRest` repository abstraction (repo hosts, stanzas, delta restore) that DBAs already know, at the cost of `pgBackRest` being version-locked to the operand image.

5. Connection pooling

CNPG provides pooling through the dedicated **Pooler CRD** — a separate, scalable Deployment of `PgBouncer` pods sitting in front of the `rw` (or `ro`) service, requiring `PgBouncer` ≥1.19 for the `auth_dbname`/`auth_query` integration (the operator provisions a `cnpg_pooler_pg_bouncer` role). Pooler config is declarative (`.spec.pg_bouncer.parameters`), supports session/transaction modes, ships a Prometheus exporter (`cnpg_pg_bouncer_*`) on port 9127, and supports PAUSE/RESUME. 1.28 added custom `PgBouncer` TLS (client-to-pooler and pooler-to-server) and TLS cipher/version controls. **Percona** integrates `PgBouncer` directly in the `PerconaPGCluster` spec (`spec.proxy.pgBouncer`) as an operator-managed component with replica instances for HA, transaction-level pooling, and read balancing — configured inline rather than as a separate CRD. [CloudNativePG + 2](#)

6. Replication topologies

CNPG replica clusters and distributed topology. A replica cluster is a separate `Cluster` in continuous recovery, fed by WAL shipping from an object store and/or streaming (cascading) from a source. The **Distributed Topology** (1.24+) spans multiple Kubernetes clusters/regions and uses `primary`, `source`, and `self` fields plus `externalClusters`. Its signature capability is **controlled switchover across data centers via demotion/promotion tokens**: demoting the current primary emits a **demotion token** (in status), which is supplied as a **promotion token** to the designated replica; the new primary waits until the designated primary has replayed WAL up to the token's LSN, then switches timelines — a zero-data-loss, fully declarative, GitOps-friendly cross-region switchover without re-cloning the former primary. Cross-cluster *automatic* failover is explicitly not done (CNPG has no authority beyond one Kubernetes cluster); it must be manual or delegated. CNPG also supports

delayed replica clusters. **Percona standby clusters** are configured via `(spec.standby)` (enabled, host, port, or a pgBackRest repo), supporting repo-based and streaming standbys across namespaces/clusters, with manual promotion — the same Crunchy standby mechanism used in the Crunchy→Percona migration playbooks.

Logical replication. CNPG provides **declarative** logical replication via the **Publication** and **Subscription** CRDs (and the **Database** CRD for declarative database/extension/schema/FDW management, extended in 1.28 with `(.spec.fdw)`/`(.spec.servers)`). This is also the basis of CNPG's online major-upgrade/migration path. Percona relies on PostgreSQL's native logical replication configured through Patroni parameters (its default `(wal_level)` is `(logical)`, now tunable to `(replica)` in 2.9.0) but does not expose first-class Publication/Subscription CRDs. [Percona](#) [GitHub](#)

7. Upgrades

Minor versions. Both do rolling updates. CNPG's `(primaryUpdateStrategy)`/`(primaryUpdateMethod)` (`(switchover)` default, or `(restart)`) governs primary handling; 1.28 allowed simultaneous image + config change with `(restart)`. Percona applies rolling updates through the Crunchy/Patroni machinery.

Major versions. CNPG offers three paths: **offline** via `(bootstrap.initdb.import)` (`(pg_dump/restore)`), **online** via native logical replication (Publication/Subscription, cutover in seconds), and **in-place** `(pg_upgrade)` (introduced ~1.26, with automated rollback when replicas are present; `(BackoffLimit=0)` on the upgrade job prevents pointless retries, and reverting the image cleanly restarts on the old version). 1.29+ even supports in-place upgrades to PG19 with ImageVolume extensions mounted side-by-side. Percona uses the **PerconaPGUpgrade** CRD, which names source/target versions and component images and runs `(pg_upgrade)` via the `(percona/percona-distribution-postgresql-upgrade)` image; the cluster is brought down for the upgrade window (this is Crunchy's `(PGUpgrade)` machinery). 2.9.0 graduated major upgrades to GA. **Operator upgrades:** CNPG follows semver within API v1, supports in-place operator upgrades and rolling instance-manager updates, and recommends not skipping minors. Percona requires upgrading the CRD (compatible with the last three minors) then the operator deployment, sequentially; the 3.0.0 upgrade performs a one-time automatic CRD-rename migration (with a 1–2 minute pgBackRest disruption while certificates propagate). [GitHub + 3](#)

8. Extensibility

CNPG-I is CNPG's defining extensibility mechanism: a gRPC-based interface (inspired by CSI) that lets plugins hook cluster-lifecycle points (backup, recovery, sub-resource reconciliation) without forking the operator. Plugins can run as sidecars (Unix-domain-socket discovery under `(PLUGIN_SOCKET_DIR)`, default `(/plugin)`) or as their own Deployments (mTLS over the network, cert-manager recommended). The **Barman Cloud Plugin** is the reference implementation. CNPG also embraces the Kubernetes **ImageVolume** feature (PG18 + K8s 1.35, or feature-gated on 1.33/1.34) to deliver extensions as OCI image volumes at runtime — declarative extension add/remove without changing the base image or breaking immutability. **Percona** loads extensions via a custom **extension-loader sidecar** and packages them in Percona Distribution for PostgreSQL images (2.6.0 added built-in pgvector); 3.0.0's community images let users request/contribute extensions (TimescaleDB, Citus shipped first). Percona also supports arbitrary sidecars (with volume/secret/configmap mounts) on Postgres, pgBouncer, and pgBackRest repo-host pods. Percona has no gRPC plugin framework equivalent to CNPG-I. [Percona](#) [Percona](#)

9. Observability

CNPG embeds a Prometheus exporter in the instance manager (metrics on port **9187**), supports **declarative custom metrics** defined in ConfigMaps/Secrets and referenced by the Cluster, ships a community Grafana dashboard, and writes all container logs as structured **JSON to stdout** for aggregation (EFK, Loki, etc.). **Percona** uses the pgMonitor-derived exporter stack and integrates deeply with **PMM (Percona Monitoring and Management)** — 2.7.0 added native PMM3 support (PMM2 deprecated in 2.9.0), Patroni metrics are sent to PMM, and `(pg_stat_monitor)` provides query analytics (disabled by default from 2.9.0 for memory safety). Observability is delivered via a PMM client sidecar rather than a native operator exporter. [CloudNativePG](#) [GitHub](#)

10. Security model

Both generate an internal CA and TLS certificates automatically and both integrate with **cert-manager**. CNPG generates and rotates its own PKI (certs handled in `(pkg/certs)`), supports custom certs and cert-manager, and (1.30) added SCRAM-SHA-256 password-encoding and search_path pinning security fixes. Percona creates a **per-cluster root CA** (since 2.5.0; earlier a shared CA), supports operator-generated certs, custom certs, or cert-manager (2.9.0 automates issue/renew/rotate), and a `(tlsOnly)` option to force encrypted channels. Both default authentication to **scram-sha-256**. **pg_hba:** CNPG manages it declaratively (including LDAP stanzas and, newly, dynamic `(podSelectorRefs)` that resolve client pod IPs by label); Percona appends rules via `(patroni.dynamicConfiguration.postgresql.pg_hba)` and per-user `(pg_hba)` entries, and 2.9.0 added first-class LDAP. Both run non-root and support OpenShift (Percona via OLM/certified operators with OperatorGroup scoping fixed in 3.0.0; CNPG's community project does not officially test OpenShift but EDB ships a certified operator). **Image provenance:** CNPG signs all images and ships SBOM + provenance attestations per architecture (distroless operator image, minimal Debian-slim operand); Percona 3.0.0 emphasizes signed, freely redistributable images and CVE-scanning alignment between runtime and upgrade images. RBAC: CNPG creates ServiceAccount/Role/RoleBinding per cluster; Percona uses Role (namespace) or ClusterRole (cluster-wide) modes. [Percona](#)

11. PostgreSQL configuration management

CNPG exposes `postgresql.conf` via `spec.postgresql.parameters`, classifies parameters by reload-vs-restart, enforces certain fixed/enforced parameters, coordinates cross-instance ordering for replication-sensitive parameters, and (1.29) validates parameter keys at the webhook to prevent injecting arbitrary directives. Roles/users are managed via the `spec.managed.roles` stanza (declarative role lifecycle, password Secrets, `validUntil`, `inRoles`, reverting drift) and, from 1.30, the standalone **DatabaseRole** CRD; databases/extensions/schemas/FDWs via the **Database** CRD. **Percona** passes PostgreSQL options through **Patroni dynamic configuration** (`patroni.dynamicConfiguration.postgresql.parameters`) — applied without validation (an invalid config can break the cluster), with `postmaster`-context changes triggering a rolling restart. Users are defined in `spec.users` with generated Secrets (`<cluster>-pguser-<user>`), custom `pg_hba` rules per user, and options. Percona's model is closer to "hand Patroni the knobs"; CNPG's is a validated, operator-mediated, drift-correcting model.

12. Storage management

CNPG manages PVCs directly (no StatefulSet volumeClaimTemplates), supports separate **WAL volumes**, declarative **tablespaces including temporary tablespaces**, local persistent volumes with PVC templates, PV reuse, volume expansion, and a `scale` subresource compatible with HPA/VPA. **Percona** manages storage through the Crunchy StatefulSet `volumeClaimTemplates`, supports separate WAL PVCs, and (2.8.x) auto-detects PVC pressure to trigger resize (auto-growable disk feature gate). Both support per-instance storage classes; Percona's multi-instance-set model allows staged storage migration by adding a new instance set with a different storage class.

13. Multi-tenancy, scheduling and topology

Both support pod anti-affinity, topology spread constraints, node selectors, and tolerations. CNPG schedules primary and replicas with anti-affinity defaults and can constrain synchronous-replica election by node label (`syncReplicaElectionConstraint`, e.g., across AZs); it manages PodDisruptionBudgets and supports `enablePDB`. Percona/PGO uses node affinity to spread instances across worker nodes and manages PDBs through the Crunchy layer; its multiple-instance-sets construct also serves multi-topology needs. Percona's reconciler concurrency (`MaxConcurrentReconciles`) matters for multi-cluster scale (backups/restores are CRs processed by the same controller), a documented "reduce the blast radius" consideration.

14. Code-level comparison

Both are Go, both use **controller-runtime**. CNPG layout: `cmd/manager` (entrypoint), `internal/` (controllers, `internal/cmd/manager/controller`, `internal/webhook/v1`), `pkg/` (`pkg/certs`, etc.), and `api/v1` (`groupName=postgresql.cnpg.io`, kubebuilder-generated). CNPG **uses admission webhooks** (validating/mutating on Cluster, e.g., `internal/webhook/v1/cluster_webhook.go`; port 9443) and increasingly CEL validation rules (e.g., immutable `cluster` reference on Database/Pooler/Publication/Subscription/ScheduledBackup). Its e2e suite is **Ginkgo-based on kind** CI; the operator implements Level V "Auto Pilot" capabilities. (DeepWiki)

Percona layout is a layered fork: Percona code (`percona/controller/pgcluster`, `percona/controller/pgbackup`, `percona/naming`, `percona/k8s`) and `pkg/apis/pgv2.percona.com/v2` sit atop the embedded Crunchy (`internal/controller/postgrescluster` and `pkg/apis/postgres-operator.crunchydata.com/v1beta1`). The central translation is **PerconaPGCluster.ToCrunchy(ctx, postgresCluster, scheme)** in `perconapgcluster_types.go`, which converts the Percona CR into a Crunchy `PostgresCluster` that the embedded reconciler owns (via owner references); the `PGClusterReconciler` struct even holds a `CrunchyController controller.Controller` reference. Percona uses controller-runtime v0.21.0 (Kubernetes libraries ~v0.33.2), server-side apply (`client.FieldOwner`), and the codebase was moved to Go 1.26. **Percona does not use admission webhooks** — consistent with Crunchy PGO v5's design; defaulting is via `PerconaPGCluster.Default()` and validation lives in the reconcile loop. Finalizers (user-facing form): `finalizers.percona.com/delete-ssl`, `finalizers.percona.com/delete-pvc`, `finalizers.percona.com/delete-backups` (plus internal `internal.percona.com/*` finalizers for watchers/backups/restores). Status uses `+kubebuilder:subresource:status` with fields like `status.state`, `status.host`, `status.postgres.ready`, `status.pgouncer.ready`, and `status.observedGeneration` (K8SPG-730). Testing is **layered**: Go unit/controller tests with Ginkgo/Gomega/testify + envtest, and **bash + KUTTL** e2e tests (`e2e-tests/functions`), KUTTL step YAML, orchestrated by Jenkins on GKE.

15. Community and ecosystem

CNPG is the most-starred PostgreSQL operator; as of mid-2026 the CNCF/LFX metrics report ~3,571 GitHub stars and rapid contributor growth (EDB and press have previously cited "5,000+ stars," reflecting counting differences between the CNCF landscape and the raw repo). Its own past-year commit activity is high, and the maintainer candidly notes in his May 2026 comparison that "174 open pull requests is a high number, and it reflects a genuine challenge: the project is growing faster than our current capacity to review and merge," attributing part of that to AI-generated contributions. Notable adopters include IBM, Google Cloud, Microsoft Azure, and (per KubeCon talks) HSBC and GEICO. CNPG supports the PGDG-supported PostgreSQL versions (18.4 default in latest 1.30/1.29 builds) across amd64/arm64. Percona's repo is smaller (~376 stars, ~78 forks at 3.0.0) but backed by Percona's commercial support, Everest UI, and PMM ecosystem; supported PostgreSQL versions in 2.9.0 are 14.22, 15.17, 16.13, 17.9, 18.3 (2.8.1 spanned 13–18; 13 dropped in 2.9.0). Commercial support: EDB (and pgEdge and others)

for CNPG; Percona subscriptions for Percona. Documentation quality is strong on both sides; CNPG's is unusually deep on architecture rationale, Percona's on operational playbooks and migration. [GitHub](#)

16. Decision framework

Recommend CloudNativePG when: the team values a Kubernetes-native control plane with the fewest moving parts inside the database pod (no Patroni, no Python runtime, no etcd); wants the smallest, fully-open, signed operand images with SBOM/provenance and CRA-relevant security artifacts; needs **declarative multi-region DR** with promotion/demotion tokens and quorum-based failover; wants **declarative logical replication** (Publication/Subscription) and declarative databases/roles; values plugin extensibility (CNPG-I) and OCI image-volume extensions; or wants vendor-neutral CNCF governance. CNPG's in-place/online major-upgrade options are more flexible than Percona's offline [pg_upgrade](#).

Recommend Percona/PGO when: the team has deep **Patroni** and **pgBackRest** operational experience and wants to reuse it; is migrating from or coexisting with a **Crunchy PGO** estate (the hard fork makes migration near-trivial — same Patroni/pgBackRest/PgBouncer, standby-cluster or backup-restore paths); prefers the mature, battle-tested Patroni consensus model and the pgBackRest repository abstraction (incremental/differential backups, delta restore); needs **Percona commercial support**, PMM integration, or Percona's TDE/[pg_tde](#) and [pg_stat_monitor](#) value-adds; or is standardized on **OpenShift via OLM** with certified operators and OperatorGroup scoping.

Migration between the two is a full logical or dump/restore migration (there is no shared on-disk or CRD format); CNPG's [initdb.import](#) and Subscription-based online import are the natural inbound paths from a Percona/Patroni cluster, while Percona→Crunchy is trivial and Crunchy→Percona is a near-zero-downtime standby-cluster operation.

Thresholds that would change the recommendation: if CNPG advances to CNCF Incubating/Graduated and closes its distributed-topology automatic-failover gap, the "maturity/consensus" argument for Patroni weakens further; conversely, if a team's entire SRE runbook, monitoring, and muscle memory are Patroni/pgBackRest, the retraining cost can outweigh CNPG's architectural elegance. For air-gapped environments, both work, but CNPG's image-volume extension model and Percona's community-image/self-built-registry model are the respective knobs to validate first.

Recommendations

1. **Default to CloudNativePG for greenfield, Kubernetes-native deployments** where you want minimal in-pod machinery, plugin extensibility, and declarative multi-cluster DR — especially if CNCF governance and fully-open signed images matter for procurement/compliance.
2. **Choose the Percona operator if you already run Crunchy PGO or Patroni/pgBackRest** in production and want to preserve that operational model with a fully-open, freely-redistributable stack and commercial support.
3. **Pilot both on a staging cluster** (Percona 3.0.0's API-group rename now allows Percona and Crunchy to coexist; CNPG installs cleanly alongside either). Benchmark failover time, synchronous-replication durability behavior, and restore time for your dataset size.
4. **For synchronous, zero-RPO requirements**, prototype CNPG's [dataDurability: required](#) + [failoverQuorum](#) against Percona's Patroni [synchronous_mode_strict](#); verify write-hang behavior matches your SLA.
5. **For DR across regions**, prototype CNPG's distributed topology with promotion tokens vs. Percona standby clusters; note CNPG's documented cross-cluster automatic-failover limitation and plan manual/federated promotion accordingly.
6. **Revisit the decision** if CNPG reaches CNCF Incubating (governance signal) or if your team's Patroni/pgBackRest expertise changes the total cost of ownership calculus.

Caveats

- Version numbers are as of July 2026: CNPG 1.30/1.29/1.28 are the live series; Percona 2.9.0 and 3.0.0 are the current releases. Both projects ship frequently — verify exact minor/patch and PostgreSQL operand versions before deployment. The in-tree Barman Cloud removal target has been pushed multiple times (1.29 → 1.30 → 1.31 in successive notes); confirm the current target before planning a migration off it.
- The most detailed public architectural comparison (Bartolini, May 2026) is authored by a CNPG co-founder and is explicitly non-neutral; its factual claims about Crunchy/Percona images and upgrades were cross-checked against Percona's own documentation where possible.
- Some source-code line numbers for the Percona repo derive from a June 2025 DeepWiki snapshot; file paths, package names, and function signatures are stable, but line ranges may have shifted on current [main](#). The "Percona uses no admission webhooks" finding is a strong inference from the inherited Crunchy design and absence across indexed sources, not a single quoted file — `grep (deploy/bundle.yaml)/(config/)` for [WebhookConfiguration](#) to confirm.
- The CNCF Sandbox acceptance date is reported as 21 January 2025 on the CNCF project page but 15 January 2025 in some EDB/Bartolini posts; the discrepancy is minor and does not affect the analysis.

- CNPG's community project does not officially test Red Hat OpenShift; OpenShift users should use EDB's certified operator. Percona explicitly supports OpenShift via OLM.
- GitHub star/contributor counts and open-PR figures are point-in-time signals, not measures of production maturity.